GENERAL EDUCATION DEPARTMENT • GOVT. OF KERALA

# GHSS Student Portal

## Complete Architecture, Source Code & Engineering Blueprint

A mission-critical School ERP and Academic Lifecycle Management System engineered for **GHSS Rajakkad (School Code: 06042)**, aligned with the **Kerala Sampoorna 2.0** statutory education ecosystem. Built on high-performance modern web standards with React 19, Node.js 24, Express 5, MongoDB / Mongoose, and Cloud-Native AWS S3 Storage.

**APPLICATION ECOSYSTEM**
MERN Stack (MongoDB, Express 5, React 19, Node 24)

**AUTHENTICATION ENGINE**
Dual Auth (JWT + Google OAuth 2.0) & Brute-Force Lockout

**ACADEMIC STRUCTURE**
26 Divisions across 4 Wings (LP, UP, High School, Plus Two)

**SECURITY & STATUTORY KYC**
Aadhaar/PAN Encryption, OTP Verification, S3 Dual-Storage

**CLIENT BUILD & TOOLING**
Vite 8 • React Router v7 • Axios • Google OAuth SDK

**REGULATORY COMPLIANCE**
Kerala Pareeksha Bhavan, SCERT & Sampoorna Standards

**Project Repository:** `ghss-student-portal`

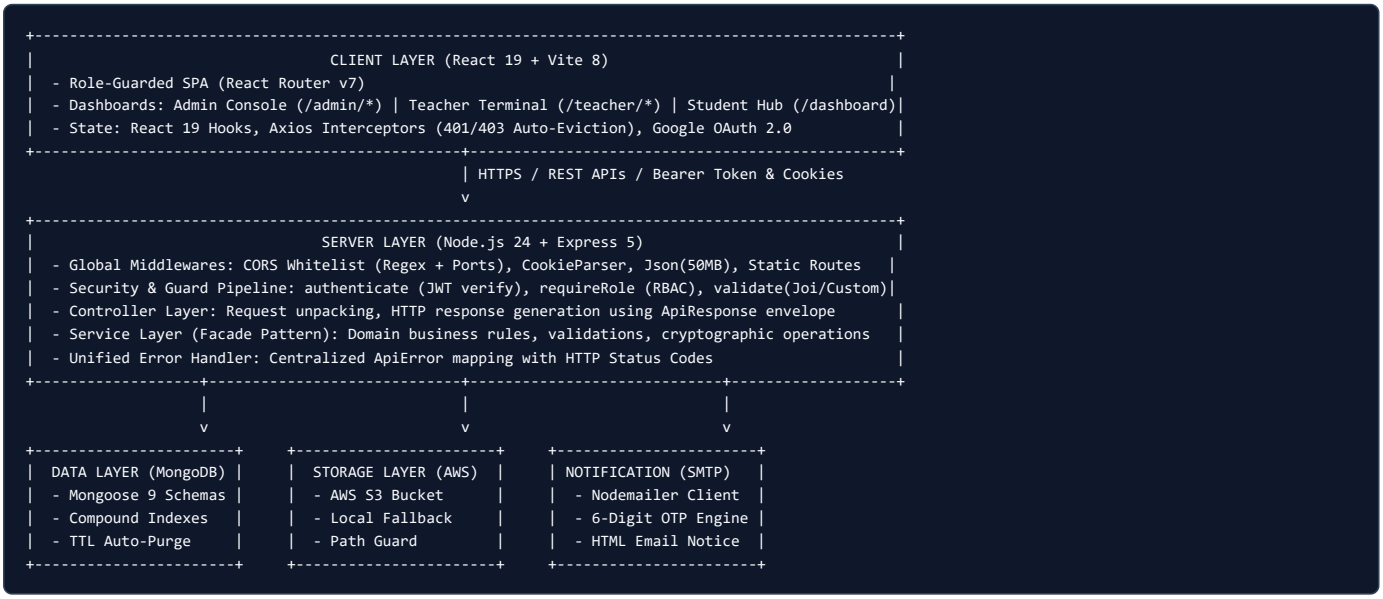
Document Generated: November 2026 • Production Release v1.0.0

# 1. Executive Summary & System Architecture

The **GHSS Student Portal** is an integrated, institutional-grade academic operating system engineered to administer the entire lifecycle of students, teachers, and administrative governance for Government Higher Secondary Schools. Built to replace fragmented paper rosters and disparate software tools, it strictly adopts the administrative norms laid out by the Kerala General Education Protection Mission and the Kerala State IT@School / Sampoorna framework.

## 1.1 Multi-Tier System Architecture

The portal is constructed following a decoupling philosophy where the frontend client and backend server communicate strictly via a RESTful API with standardized JSON response envelopes.




## 1.2 Core Design Principles & Patterns

**PATTERN** **Controller-Service-Repository**

Controllers strictly handle HTTP protocol concerns (headers, parameters, and calling the standardized `ApiResponse`). All business calculations, database mutations, and external calls reside in isolated, reusable static classes in `src/services/`.

**PATTERN** **Service Facade Architecture**

Complex domains like Student administration are split into specialized modules (`studentAdmin`, `studentExams`, `studentFees`, `studentMetrics`) while unified behind the main `StudentService` facade for zero controller refactoring.

**DEFENSE** **Dual-Storage Engine with Fallback**

Statutory document uploads transparently write to AWS S3 if active credentials exist; otherwise, they seamlessly fall back to local disk storage with cryptographic random names and path-traversal guards.

**SECURITY** **Centralized Operational Error Model**

Custom `ApiError` class extends JavaScript's `Error` to carry HTTP status codes, isOperational flags, validation errors, and lockout states. The global error handler converts unhandled exceptions into clean JSON responses without leaking system stack traces.

## 2. Complete Codebase & Directory Architecture

The project is structured as a coordinated full-stack repository with root package scripts orchestrating both sub-applications concurrently.

## 2.1 Annotated Repository Tree

```
School  portal/
├── package.json                        # Root workspace scripts (npm run client, server, seed)
│
├── client/                             # Vite + React 19 Frontend Application
│   ├── .env                            # VITE_API_URL, VITE_GOOGLE_CLIENT_ID
│   ├── index.html                      # Single-page HTML mounting document
│   ├── vite.config.js                  # Vite bundler configuration & React 19 plugins
│   ├── src/
│   │   ├── main.jsx                    # Application bootstrap & GoogleOAuthProvider mount
│   │   ├── App.jsx                     # Client router (PublicRoute / ProtectedRoute guards)
│   │   ├── api/
│   │   │   ├── axiosclient.js          # Axios instance with baseURL, credentials, interceptors
│   │   │   └── authapi.js              # API service methods for login and token refresh
│   │   ├── components/
│   │   │   ├── admin/                  # Administrative modular UI components
│   │   │   │   ├── AdminHeader.jsx     # Top navigation with avatar, profile, and quick actions
│   │   │   │   ├── AdminSidebar.jsx    # Division muster and navigation menu
│   │   │   │   ├── attendance/         # AttendanceRegisterModal (bulk day-by-day marking)
│   │   │   │   ├── exams/              # ExamScheduleModal, ExamSeatingModal, ExamStatsCards
│   │   │   │   ├── payments/           # PaymentFormModal, PaymentReceiptModal, PaymentStatsCards
│   │   │   │   ├── students/           # StudentFormModal, StudentDossierModal, StudentStatsCards
│   │   │   │   └── teachers/           # TeacherFormModal, TeacherDossierModal, TeacherAllotmentModal
│   │   │   ├── common/
│   │   │   │   └── EmailOtpVerification.jsx # 6-digit OTP verification input component
│   │   │   ├── student/                # Student portal tabs (Overview, Attendance, Fees, Timetable)
│   │   │   └── teacher/                # Teacher portal tabs (Academics, Marks, Attendance, Schedule)
│   │   ├── Styles/                     # Dedicated CSS stylesheets (AdminDash, StudentDash, etc.)
│   │   ├── utils/                      # Client validation and formatting helpers
│   │   │   ├── ageValidator.js         # Admission & faculty age limit calculators
│   │   │   ├── auth.js                 # Session storage, token decode, role dashboard paths
│   │   │   ├── classData.js            # 26 Consolidated academic divisions & wing registry
│   │   │   ├── documentUrl.js          # S3 vs Local Uploads URL resolver
│   │   │   ├── examDateValidator.js    # Exam schedule formatting and chronology checks
│   │   │   ├── gradeCalculator.js      # Kerala 9-point Pareeksha Bhavan grade calculator
│   │   │   └── nameValidator.js        # Strict human name regex validator
│   │   └── views/                      # Main page containers (Auth, Admin, Teacher, Student)
│
└── server/                             # Node.js 24 + Express 5 Backend REST Engine
    ├── .env                            # PORT, MONGO_URI, JWT_SECRET, AWS credentials, SMTP
    ├── src/
    │   ├── server.js                   # Server startup script, connects DB then listens
    │   ├── app.js                      # Express app configuration, CORS, static routes, 404
    │   ├── config/
    │   │   └── db.js                   # MongoDB connection with Mongoose configuration
    │   ├── constants/
    │   │   └── enums.js                # UserRole, AttendanceStatus, Gender, Department, LEAVE_LIMITS
    │   ├── models/                     # Mongoose Database Schemas
    │   │   ├── user.js                 # User credentials, lockout flags, reset OTPs, roles
    │   │   ├── Student.js              # Comprehensive student admission & KYC legal records
    │   │   ├── Teacher.js              # Teacher cadre, PEN number, periods, qualifications
    │   │   ├── Attendance.js           # Daily division muster roll with compound unique index
    │   │   ├── Exam.js                 # Board/terminal exam schedules, QP codes, seating plans
    │   │   ├── FeePayment.js           # Fee collection ledger, receipt numbers, payment modes
    │   │   ├── InternalMark.js         # Continuous Evaluation (CE) marks & grade records
    │   │   └── EmailVerification.js    # Temporary OTP documents with MongoDB TTL auto-purge
    │   ├── controllers/                # HTTP Request/Response Controllers
    │   │   ├── admin.controller.js     # Aggregated entrypoint for administrative actions
    │   │   ├── admin/                  # Modular sub-controllers (Students, Teachers, Exams, Fees)
    │   │   ├── auth.controller.js      # Login, Google OAuth, password reset, OTP verification
    │   │   ├── student.controller.js   # Student portal endpoints (Metrics, Exams, Fee history)
    │   │   ├── teacher.controller.js   # Teacher portal endpoints (Muster, Marks, Timetable)
    │   │   └── upload.controller.js    # Multer file receiver and S3 upload handler
    │   ├── services/                   # Domain Business Logic & Facades
    │   │   ├── auth.service.js         # Password compare, account lockout, JWT generation
    │   │   ├── student.service.js      # Facade over studentAdmin, studentExams, studentFees
    │   │   ├── teacher.service.js      # Teacher CRUD, cadre statistics, timetable periods
    │   │   ├── exam.service.js         # Timetable scheduling, invigilator PEN lookup
    │   │   ├── payment.service.js      # Financial accounting, receipt generation, dues computation
    │   │   ├── otp.service.js          # Cryptographic OTP generator, verification and consumption
    │   │   ├── email.service.js        # Nodemailer SMTP transporter and HTML email templates
    │   │   └── s3.service.js           # AWS SDK v3 PutObject with local disk fallback & path guard
    │   ├── middleware/                 # Express Middlewares
    │   │   ├── auth.middleware.js      # JWT extraction and validation
    │   │   ├── role.middleware.js      # RBAC guards (requireAdmin, requireTeacher, requireStudent)
    │   │   ├── error.middleware.js     # Global centralized error handler
    │   │   ├── upload.middleware.js    # Multer memory storage and file type filters
    │   │   └── validate.middleware.js  # Request schema validator
    │   ├── utils/                      # Shared server utilities
    │   │   ├── ApiError.js             # Standard operational error class
    │   │   ├── ApiResponse.js          # Uniform HTTP response helper
```

```
│   │   └── asyncHandler.js          # Asynchronous promise handler eliminating try/catch blocks
│   └── Seed/                        # Database Seeding Scripts (seedData.js, seedAdmin.js, etc.)
```

# 3. Security, Authentication & Access Control

Security is woven through every layer of the GHSS Student Portal. From cryptographic password hashing to brute-force protection and role-based route gating, the system ensures zero unauthorized data access.

## 3.1 Dual-Engine Authentication & Brute-Force Defense

Authentication is provided through either traditional local credentials (Email & Password) or Google OAuth 2.0 single sign-on. The local login workflow features an aggressive anti-brute-force mechanism:

```
// server/src/services/auth.service.js (Lockout & Security Logic)
const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) {
  const attempts = (user.loginAttempts || 0) + 1;
  const willLock = attempts >= 5;
  const lockUntil = willLock ? new Date(Date.now() + 15 * 60 * 1000) : null;

  await User.findByIdAndUpdate(user._id, {
    $set: { loginAttempts: attempts, lockUntil: lockUntil }
  });

  if (willLock) {
    const lockError = new ApiError(
      403,
      "You have entered wrong details 5 times. Your account has been temporarily blocked for 15 minutes."
    );
    lockError.isLocked = true;
    lockError.remainingMinutes = 15;
    throw lockError;
  }
  throw ApiError.unauthorized(`Invalid credentials. (${5 - attempts} attempts left before lockout)`);
}

// Reset attempts on successful authentication
await User.findByIdAndUpdate(user._id, { $set: { loginAttempts: 0, lockUntil: null } });
```

> **Account Security Protocol:** After exactly 5 consecutive failed login attempts, the target account is locked for 15 minutes. Even if the correct password is submitted during this window, the API strictly rejects authentication with HTTP 403 and informs the user of the exact minutes remaining.

## 3.2 Session Lifecycle & Auto-Eviction

When a user logs in, the server signs a JSON Web Token (JWT) containing their `id`, `email`, `name`, and `role`. On the client side, `src/utils/auth.js` manages the session with a strict **1-Hour Absolute Expiration Window**:

- **Storage:** Token, User Metadata, and Expiration Timestamp are persisted in `localStorage`.
- **Session Verification:** Every route navigation via `ProtectedRoute` verifies if `Date.now() > auth_expires_at`.
- **Auto-Eviction:** When expired, `clearAuthSession()` immediately clears all stored keys and redirects to `/login?session=expired`.
- **Axios Interceptor:** If the backend returns HTTP 401 Unauthorized, the client interceptor automatically initiates session cleanup.

## 3.3 6-Digit OTP Verification Engine

To prevent invalid or fraudulent admissions and faculty registrations, the system employs an OTP verification gate before new records can be inserted into the database.

| Workflow Purpose | Trigger Endpoint | Verification Endpoint | Security Enforcement |
|---|---|---|---|
| **Student Admission** | `POST /api/auth/send-registration-otp` | `POST /api/auth/verify-registration-otp` | Admin cannot submit student record until email OTP is marked verified in DB. |
| **Teacher Registration** | `POST /api/auth/send-registration-otp` | `POST /api/auth/verify-registration-otp` | Faculty record creation checks `OtpService.isEmailVerified` prior to save. |
| **Password Recovery** | `POST /api/auth/forgot-password` | `POST /api/auth/verify-otp` | 10-minute validity. Enforces OTP check before `/reset-password` is accepted. |

# 4. Database Schemas & Data Models (MongoDB / Mongoose)

The data model architecture is carefully designed to accommodate the complex requirements of the Kerala General Education Department, including Sampoorna IDs, Permanent Employee Numbers (PEN), statutory ID proof documents, and compound-indexed muster rolls.

## 4.1 Core Entity Models

### 1. User Model (`models/user.js`)

Acts as the centralized authentication identity for all actors in the system.

```
{
  name: { type: String, trim: true, validate: validateHumanName },
  username: { type: String, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, minlength: 6 },
  role: { type: String, enum: ["admin", "teacher", "student"], default: "student" },
  isGoogleAuth: { type: Boolean, default: false },
  avatarUrl: { type: String, default: null },
  isActive: { type: Boolean, default: true },
  loginAttempts: { type: Number, default: 0 },
  lockUntil: { type: Date, default: null },
  resetPasswordOtp: { type: String, default: null },
  resetPasswordExpires: { type: Date, default: null }
}
```

**Pre-Validate Hook:** Automatically normalizes `name` and `username` so neither is ever undefined. Converts legacy `role: "user"` to `"student"`.
**Pre-Save Hook:** Automatically runs `bcrypt.hash(this.password, 10)` if modified.

### 2. Student Model (`models/Student.js`)

Maintains the statutory identity, admission ledger, and academic tracking for every enrolled pupil.

```
{
  userId: { type: ObjectId, ref: "User", sparse: true },
  rollNumber: { type: Number, required: true },
  admissionNumber: { type: String, unique: true, sparse: true }, // e.g. "ADM-14920"
  sampoornaId: { type: String, trim: true },                     // e.g. "SMP-06042-881"
  name: { type: String, required: true, validate: validateHumanName },
  email: { type: String, lowercase: true, sparse: true },
  classDivision: { type: String, required: true, default: "Class 10-A" },
  gender: { type: String, enum: ["Male", "Female", "Other", "Boy", "Girl"] },
  dob: { type: String, trim: true },                             // "YYYY-MM-DD"
  category: { type: String, default: "General" },                 // General, OBC, SC, ST, OEC
  aadhaarNumber: { type: String, trim: true, sparse: true },     // Masked in UI views
  birthCertificateNumber: { type: String, trim: true },
  idProofUrl: { type: String, trim: true },                      // S3 / Uploads path
  idProofType: { type: String, default: "Aadhaar Card" },
  bloodGroup: { type: String, enum: ["A+", "A-", "B+", "B-", "O+", "O-", "AB+", "AB-", "Unknown"] },
  fatherName: { type: String, validate: validateHumanName },
  motherName: { type: String, validate: validateHumanName },
  guardian: { type: String, validate: validateHumanName },
  parentContact: { type: String, default: "+91 94470 00000" },
  address: {
    houseName: String, street: String, postOffice: String,
    city: String, district: { type: String, default: "Idukki" },
    state: { type: String, default: "Kerala" }, pincode: String
  },
  monthlyLeavesTaken: { type: Number, default: 0 },
  overallAttendance: { type: String, default: "98.4" }
}
```

### 3. Teacher Model ( `models/Teacher.js` )

Represents government-sanctioned teaching faculty, designations, cadre levels, and weekly workload allotments.

```
{
  userId: { type: ObjectId, ref: "User", sparse: true },
  name: { type: String, required: true, validate: validateHumanName },
  pen: { type: String, trim: true, sparse: true },           // Permanent Employee Number (e.g. "849201")
  penNumber: { type: String, trim: true, sparse: true },     // e.g. "PEN-849201"
  department: { type: String, default: "Mathematics" },
  designation: { type: String, default: "HSST" },            // HST, HSST, LPSA, UPSA
  assignedClass: { type: String, default: "Class 10-A" },    // Sanitized to remove "Mentor" suffixes
  employmentType: { type: String, default: "Permanent / Sanctioned" },
  periodsPerWeek: { type: Number, default: 24 },
  qualification: { type: String, default: "M.Sc. Mathematics, B.Ed, SET" },
  dob: { type: String },                                     // Validated for 20-56 age boundaries
  status: { type: String, enum: ["Active", "In Class", "On Duty", "On Leave"], default: "Active" }
}
```

### 4. Attendance Model ( `models/Attendance.js` )

Stores division-wide daily roll-call records. Features a strict **compound unique index** to guarantee that no class division can have duplicate attendance registers for the same calendar date.

```
const singleRecordSchema = new mongoose.Schema({
  studentId: { type: ObjectId, ref: "Student", required: true },
  rollNumber: { type: Number, required: true },
  name: { type: String, required: true },
  status: { type: String, enum: ["Present", "Absent", "On Leave", "Leave", "Half Day"], default: "Present" },
  remarks: { type: String, default: "" }
}, { _id: false });

const attendanceSchema = new mongoose.Schema({
  date: { type: String, required: true },                    // "YYYY-MM-DD"
  classDivision: { type: String, required: true },           // e.g. "Class 10-A"
  records: [singleRecordSchema],
  totalPresent: { type: Number, default: 0 },
  totalAbsent: { type: Number, default: 0 },
  markedBy: { type: String, default: "kathalanx@gmail.com" }
}, { timestamps: true });

// Compound Unique Index:
attendanceSchema.index({ date: 1, classDivision: 1 }, { unique: true });
```

### 5. FeePayment Model ( `models/FeePayment.js` )

Tracks statutory student collections across all 4 school wings. Automatically generates verifiable receipt IDs formatted as `REC-2026-XXXX`.

```
{
  receiptNumber: { type: String, required: true, unique: true },
  studentId: { type: ObjectId, ref: "Student" },
  studentName: { type: String, required: true },
  admissionNumber: { type: String, required: true },
  classDivision: { type: String, required: true },
  wing: { type: String, enum: ["LP School", "UP School", "High School", "Higher Secondary"] },
  feeCategory: {
    type: String,
    enum: ["Bus Fee", "Annual Fee", "PTA Fund", "Lab & Computer Fee", "Special / Exam Fee", "Sports & Library"]
  },
  term: { type: String, enum: ["Annual", "Term 1", "Term 2", "Term 3", "Monthly Bus Pass", "One-Time"] },
  totalAmount: { type: Number, required: true, min: 0 },
  amountPaid: { type: Number, required: true, min: 0 },
  dueAmount: { type: Number, default: 0 },
  paymentDate: { type: String, required: true },
  paymentMode: { type: String, enum: ["UPI / Online (Google Pay / PhonePe)", "Net Banking", "Cash at School Counter", "Bank Challan (SBT/SBI)"] },
  status: { type: String, enum: ["Paid", "Partial", "Pending", "Overdue"], default: "Paid" }
}
```

# 5. Backend Services & Controller Layer

The server encapsulates all complex transactions inside specialized services, ensuring the HTTP controllers remain lean, declarative, and easily testable.

## 5.1 Unified Student Service & Sub-Domain Services

The `StudentService` class acts as an overarching facade over four domain-specific sub-services:

| Service Class | Key Operations | Domain Logic & Validations |
|---|---|---|
| `StudentAdminService` | • `getStudentsRecords`<br>• `createStudent`<br>• `updateStudent`<br>• `deleteStudent` | Validates human names; verifies age against class standard; verifies and consumes OTP; auto-generates Sampoorna ID and Admission No; automatically provisions linked `User` account with bcrypt-hashed default password. |
| `StudentMetricsService` | • `getStudentMetrics`<br>• `getStudentDossier` | Aggregates live attendance muster percentage; computes remaining monthly leaves (enforces max 3 days per statutory rule); compiles parent contact and address dossier. |
| `StudentExamsService` | • `getStudentExams`<br>• `generateHallTicket` | Retrieves published term timetables; resolves examination hall allocations and cool-off timings; builds printable SSLC Board Examination hall tickets. |
| `StudentFeesService` | • `getStudentFees`<br>• `getStudentReceipt` | Retrieves fee ledger entries; computes balance dues; issues formatted receipt data for immediate student download. |

## 5.2 Teacher Service & Cadre Management ( `teacher.service.js` )

Manages the school's teaching staff roster, period assignments, and class mentorships.

- **Cadre Statistics:** Automatically tallies sanctioned posts, permanent staff (HST/HSST/LPSA/UPSA), on-duty teachers, and guest faculty vacancies against statutory allotment quotas.
- **Permanent Employee Number (PEN):** Auto-generates unique 6-digit PEN identifiers (e.g. `PEN-849201`) conforming to the Kerala Government SPARK payroll format.
- **Continuous Evaluation (CE) Scoring:** Facilitates terminal marks entry for students and converts raw marks into Kerala Pareeksha Bhavan grades (A+ to E) using `calculateKeralaGrade()`.

## 5.3 Exam Scheduling Service ( `exam.service.js` )

Administers SSLC Board and terminal exam schedules. A key capability is the **Automatic Invigilator PEN Resolver**:

```
// server/src/services/exam.service.js
async function resolveInvigilatorsWithPen(invigilatorString) {
  if (!invigilatorString) return [];
  const rawNames = invigilatorString.split(/[,/;&]|□and□/i).map(s => s.trim()).filter(Boolean);
  const teachersList = await Teacher.find({}).select("name pen penNumber").lean();

  return rawNames.map((name) => {
    const clean = name.replace(/^(mrs\.|mr\.|smt\.|sri\.|dr\.)\s*/i, "").trim().toLowerCase();
    const found = teachersList.find((t) => {
      const tClean = (t.name || "").replace(/^(mrs\.|mr\.|smt\.|sri\.|dr\.)\s*/i, "").trim().toLowerCase();
      return tClean === clean || tClean.includes(clean) || clean.includes(tClean);
    });
    return { name, pen: found?.pen || found?.penNumber?.replace(/\D/g, "") || "849201" };
  });
}
```

# 6. Frontend Portals & User Experience (React 19)

The client application is built with React 19 and Vite 8, featuring modular role-specific interfaces designed with clean responsive typography, modal workflows, and zero latency state synchronization.

## 6.1 Application Route Matrix & Route Guards

All routes in `client/src/App.jsx` are shielded by high-order guard components:

```
// client/src/App.jsx
function ProtectedRoute({ children, allowedRoles }) {
  if (!isSessionValid()) {
    clearAuthSession();
    return <Navigate to="/login?session=expired" replace />;
  }

  if (allowedRoles) {
    const userRole = JSON.parse(localStorage.getItem("user") || "{}").role || "student";
    if (!allowedRoles.includes(userRole)) {
      return <Navigate to={getRoleDashboardPath(userRole)} replace />;
    }
  }
  return children;
}
```

## 6.2 The Three Role Portals

**ADMIN PORTAL** **/admin/***

**Comprehensive School Governance Suite:**

- **Admin Overview:** Live student/staff headcounts and attendance sync for 4 academic wings.
- **Division Attendance:** 26-division muster rolls with `AttendanceRegisterModal` for live roll marking.
- **Teacher Cadre:** Faculty directory, PEN tracker, qualification verification, and weekly period schedules.
- **Student Records:** Complete admission console, age verification, Aadhaar masking, and KYC dossier viewer.
- **Exam Timetables:** SSLC schedule manager, hall allocation, and room-by-room seating plan generation.
- **Fee Console:** Multi-category collections, payment receipts, and fee balance analytics.

**TEACHER PORTAL** **/teacher/***

**Class Mentor & Subject Faculty Terminal:**

- **Attendance Tab:** Division roll-call with bulk Present/Absent toggles and instant MongoDB update.
- **Academics Tab:** Class roster, student contact details, and statutory document inspections.
- **Internal Marks:** Kerala Continuous Evaluation (CE) terminal mark entry with auto-grade calculations.
- **Timetable Tab:** Weekly allotted teaching hours, classrooms, and subject schedules.
- **Faculty Profile:** Official SPARK PEN credentials, departmental cadre, and service records.

**STUDENT PORTAL** **/dashboard**

**Pupil & Parent Self-Service Hub:**

- **Overview Tab:** Live morning timetable, school announcements, attendance summary, and verified profile summary.
- **Attendance Ledger:** Monthly calendar breakdown, leave compliance monitor (alerts if approaching 3-day cap).
- **Timetable & Hall Tickets:** Term exam schedules, QP codes, and one-click printable SSLC Hall Ticket download.
- **Fee Ledger & Receipts:** Paid transaction receipts, balance dues, and instant official printable fee vouchers.

# 7. Business Logic & Regulatory Validators

The system embeds specialized domain algorithms that strictly follow Kerala Government education orders.

## 7.1 Kerala Pareeksha Bhavan 9-Point Grading Scale

Implements the standard secondary school grading system defined in `utils/gradeCalculator.js`:

| Score Range | Grade Awarded | Performance Descriptor | Qualifying Status |
|---|---|---|---|
| **90% – 100%** | A+ | Outstanding | Passed with Distinction |
| **80% – 89%** | A | Excellent | Passed with High First Class |
| **70% – 79%** | B+ | Very Good | Passed with First Class |
| **60% – 69%** | B | Good | Passed with Second Class |
| **50% – 59%** | C+ | Above Average | Passed with Third Class |
| **40% – 49%** | C | Average | Passed (Minimum Pass Standard) |
| **30% – 39%** | D+ | Marginal | Eligible for Improvement / SAY Exam |
| **Below 30%** | E | Needs Improvement | Failed / Retest Required |
| **Absent** | AB | Absent from Examination | Not Qualified |

## 7.2 Strict Human Name Validator ( `nameValidator.js` )

Enforces realistic name patterns on both frontend and backend models to reject gibberish, digits, and special characters:

```
export const validateHumanName = (name, fieldLabel = "Name") => {
  if (!name || typeof name !== "string") return `${fieldLabel} is required.`;
  const trimmed = name.trim();
  if (trimmed.length < 2) return `${fieldLabel} must be at least 2 characters long.`;
  if (trimmed.length > 70) return `${fieldLabel} cannot exceed 70 characters.`;

  // Allows letters, dots (initials), spaces, apostrophes, and hyphens
  const nameRegex = /^[A-Za-z]+([ .'-][A-Za-z]+)*[.]?$/;
  if (!nameRegex.test(trimmed)) {
    return `${fieldLabel} contains invalid characters. Numbers and special symbols are prohibited.`;
  }
  return true;
};
```

## 7.3 Kerala Academic Structure & Division Registry ( `classData.js` )

Registers the 26 academic divisions across GHSS Rajakkad's 4 distinct wings:

- **Lower Primary (LP School):** Classes 1 to 4 • 8 Divisions (1-A to 4-B) • Mentored by LPSA faculty.
- **Upper Primary (UP School):** Classes 5 to 7 • 6 Divisions (5-A to 7-B) • Mentored by UPSA faculty.
- **High School Section:** Classes 8 to 10 • 6 Divisions (8-A to 10-B) • SSLC Cadre • Mentored by HST faculty.
- **Higher Secondary Section:** Classes 11 & 12 • 6 Divisions (Science, Commerce, Humanities) • Mentored by HSST faculty.

# 8. Comprehensive REST API Endpoint Reference

All API endpoints require JSON payloads and return a standardized response envelope: `{ "success": boolean, "message"?: string, "data"?: any }`.

| Method | Endpoint Route | Security / Guard | Description & Operation |
|---|---|---|---|
| POST | `/api/auth/login` | Public (Rate-limited) | Email/Password login with 5-attempt brute-force lockout. |
| POST | `/api/auth/google` | Public | Verifies Google OAuth ID token and returns session JWT. |
| POST | `/api/auth/forgot-password` | Public | Generates and emails 6-digit password reset OTP (10 min expiry). |
| POST | `/api/auth/reset-password` | Public | Validates reset OTP and updates user's bcrypt password. |
| POST | `/api/auth/send-registration-otp` | Public | Dispatches 6-digit OTP for student admission / teacher onboarding. |
| POST | `/api/auth/verify-registration-otp` | Public | Validates registration OTP and marks email verified in DB. |
| GET | `/api/admin/overview-stats` | JWT + Admin | Aggregates school headcounts, staff on duty, and wing turnouts. |
| GET | `/api/admin/division-muster` | JWT + Admin | Returns roll-call muster status for all 26 academic divisions. |
| POST | `/api/admin/attendance/register` | JWT + Admin | Saves/updates daily class attendance muster register. |
| GET | `/api/admin/teachers` | JWT + Admin | Lists all teachers, cadre breakdown, and sanctioned vacancies. |
| POST | `/api/admin/teachers` | JWT + Admin | Enrolls faculty with verified email OTP, PEN, and age checks. |
| GET | `/api/admin/students` | JWT + Admin | Fetches student census with division filters and search queries. |
| POST | `/api/admin/students` | JWT + Admin | Admits pupil, generates Sampoorna ID, and creates user login. |
| GET | `/api/admin/exams` | JWT + Admin | Retrieves SSLC board and terminal examination schedules. |
| GET | `/api/admin/payments` | JWT + Admin | Queries fee collection ledger with status and wing filters. |
| POST | `/api/admin/payments` | JWT + Admin | Logs fee transaction and generates receipt `REC-2026-XXXX`. |
| GET | `/api/teacher/students` | JWT + Teacher | Retrieves class students list for attendance marking. |
| POST | `/api/teacher/attendance` | JWT + Teacher | Submits daily roll-call attendance register for assigned division. |
| GET | `/api/teacher/internal-marks` | JWT + Teacher | Fetches Continuous Evaluation (CE) marks table for subject. |
| POST | `/api/teacher/internal-marks` | JWT + Teacher | Saves internal marks with automated Kerala grade evaluation. |
| GET | `/api/teacher/timetable` | JWT + Teacher | Retrieves teacher's weekly subject & period schedule. |
| GET | `/api/student/dashboard-data` | JWT + Student | Delivers student dashboard metrics, notices, and leave ledger. |
| GET | `/api/student/exams` | JWT + Student | Retrieves exam timetable and hall ticket generation payload. |
| GET | `/api/student/fees` | JWT + Student | Returns student's fee payment history and outstanding balance. |
| POST | `/api/upload/avatar` | JWT Authenticated | Uploads avatar image to S3/Disk and updates User profile. |
| POST | `/api/upload/document` | JWT Authenticated | Uploads statutory KYC document (PDF/Image) to secure storage. |

# 9. Deployment, Operations & Verification Guide

The GHSS Student Portal is configured for simple local development and production deployment.

## 9.1 Environment Variable Configuration

**Server Environment ( `server/.env` )**

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/ghss_portal
JWT_SECRET=super_secret_jwt_key_2026
CLIENT_URL=http://localhost:5173

# Google OAuth 2.0 Credentials
GOOGLE_CLIENT_ID=your_google_client_id.apps.googleusercontent.com

# AWS S3 Storage (Optional: Falls back to local disk)
AWS_ACCESS_KEY_ID=your_aws_key
AWS_SECRET_ACCESS_KEY=your_aws_secret
AWS_REGION=ap-south-1
AWS_S3_BUCKET_NAME=ghss-portal-uploads

# SMTP Email Configuration (Nodemailer OTP)
EMAIL_SERVICE=gmail
EMAIL_USER=institutional_alerts@ghss.edu.in
EMAIL_PASS=your_app_specific_password
```

**Client Environment ( `client/.env` )**

```
VITE_API_URL=http://localhost:5000/api
VITE_GOOGLE_CLIENT_ID=your_google_client_id.apps.googleusercontent.com
```

## 9.2 NPM Execution & Database Seeding Scripts

| CLI Command | Working Directory | Description & Lifecycle Stage |
|---|---|---|
| `npm run client` | Root ( `/` ) | Starts the Vite React 19 development server at `http://localhost:5173`. |
| `npm run server` | Root ( `/` ) | Launches the Express 5 API server with `nodemon` hot-reloading at `port 5000`. |
| `npm run seed:admin` | Root ( `/` ) | Seeds the default institutional Administrator account into MongoDB. |
| `npm run seed:teacher` | Root ( `/` ) | Populates baseline faculty records with SPARK PEN numbers. |
| `npm run seed` | Root ( `/` ) | Executes comprehensive seed: classes, students, attendance, and fee ledgers. |
| `npm run client:build` | Root ( `/` ) | Compiles and bundles the production frontend into `client/dist`. |

## 9.3 Verification & Security Checklist

**Production Readiness Audit:**

- ✓ **No Hardcoded Passwords:** Passwords hashed with salt factor 10 using `bcryptjs`.
- ✓ **Statutory Document Protection:** Route `/uploads/documents` is guarded by the `authenticate` middleware; unauthenticated users cannot access sensitive Aadhaar/KYC files.
- ✓ **CORS Protection:** Whitelists only recognized production domains and developer ports via dynamic regex.
- ✓ **Brute-Force Guard:** Verified 5-attempt threshold triggers a 15-minute lock.
- ✓ **Auto-Purging Ephemeral Data:** MongoDB TTL index ensures unverified OTPs self-delete within 10 minutes.

— End of Architectural & Codebase Documentation • GHSS Student Portal —